```python
"""Populate shipment_database.db from the shipping department's spreadsheets.

Database schema:
    product(id INTEGER PRIMARY KEY, name TEXT UNIQUE)
    shipment(id INTEGER PRIMARY KEY, product_id -> product, quantity, origin, destination)

Spreadsheets:
    shipping_data_0.csv  one row per (shipment, product) with quantity, origin, destination.
                         Self-contained: each row becomes one shipment row.
    shipping_data_1.csv  one row per *unit* of product, tagged with a shipment_identifier.
                         Rows are grouped by (shipment_identifier, product);
                         the size of each group is the quantity.
    shipping_data_2.csv  origin and destination for each shipment_identifier in spreadsheet 1.

Usage:
    python populate_database.py            # fills an empty database
    python populate_database.py --reset    # clears both tables first, then fills them
"""

import argparse
import csv
import sqlite3
from collections import Counter
from pathlib import Path

REPO_DIR = Path(__file__).resolve().parent
DATA_DIR = REPO_DIR / "data"
DATABASE_PATH = REPO_DIR / "shipment_database.db"


def read_rows(file_name):
    """Return the rows of a spreadsheet in data/ as a list of dicts keyed by column header."""
    with open(DATA_DIR / file_name, newline="", encoding="utf-8") as csv_file:
        return list(csv.DictReader(csv_file))


class ShipmentWriter:
    """Inserts shipments, creating each product the first time its name is seen."""

    def __init__(self, connection):
        self.cursor = connection.cursor()
        # Cache name -> id so each product costs one lookup, not one per shipment.
        self.product_ids = {
            name: product_id
            for product_id, name in self.cursor.execute("SELECT id, name FROM product")
        }
        self.shipments_inserted = 0

    def product_id(self, name):
        if name not in self.product_ids:
            self.cursor.execute("INSERT INTO product (name) VALUES (?)", (name,))
            self.product_ids[name] = self.cursor.lastrowid
        return self.product_ids[name]

    def insert_shipment(self, product_name, quantity, origin, destination):
        self.cursor.execute(
            "INSERT INTO shipment (product_id, quantity, origin, destination) "
            "VALUES (?, ?, ?, ?)",
            (self.product_id(product_name), quantity, origin, destination),
        )
        self.shipments_inserted += 1


def load_spreadsheet_0(writer):
    """Each row already describes one product in one shipment: insert it as it is."""
    for row in read_rows("shipping_data_0.csv"):
        writer.insert_shipment(
            product_name=row["product"],
            quantity=int(row["product_quantity"]),
```

```
            origin=row["origin_warehouse"],
            destination=row["destination_store"],
        )


def load_spreadsheets_1_and_2(writer):
    """Combine the per-unit rows of spreadsheet 1 with the routes in spreadsheet 2."""
    # Spreadsheet 2: shipment_identifier -> (origin, destination)
    routes = {
        row["shipment_identifier"]: (row["origin_warehouse"], row["destination_store"])
        for row in read_rows("shipping_data_2.csv")
    }

    # Spreadsheet 1: one row per unit, so counting the rows for each (shipment, product)
    # gives the quantity.
    # Counter keeps first-seen order, so shipments are inserted in spreadsheet order.
    quantities = Counter(
        (row["shipment_identifier"], row["product"])
        for row in read_rows("shipping_data_1.csv")
    )

    for (shipment_identifier, product_name), quantity in quantities.items():
        if shipment_identifier not in routes:
            raise ValueError(
                f"shipment {shipment_identifier} has no origin/destination in spreadsheet 2"
            )
        origin, destination = routes[shipment_identifier]
        writer.insert_shipment(product_name, quantity, origin, destination)


def main():
    parser = argparse.ArgumentParser(
        description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter
    )
    parser.add_argument(
        "--reset", action="store_true", help="delete existing rows before loading"
    )
    args = parser.parse_args()

    connection = sqlite3.connect(DATABASE_PATH)
    try:
        # One transaction: either every spreadsheet is loaded or, on any error, nothing is.
        with connection:
            if args.reset:
                connection.execute("DELETE FROM shipment")
                connection.execute("DELETE FROM product")
            elif connection.execute("SELECT COUNT(*) FROM shipment").fetchone()[0]:
                raise SystemExit(
                    "shipment table already has rows; rerun with --reset to reload it"
                )

            writer = ShipmentWriter(connection)
            load_spreadsheet_0(writer)
            from_spreadsheet_0 = writer.shipments_inserted
            load_spreadsheets_1_and_2(writer)

        print(f"Inserted {from_spreadsheet_0} shipments from spreadsheet 0 and "
              f"{writer.shipments_inserted - from_spreadsheet_0} from spreadsheets 1 and 2 "
              f"({len(writer.product_ids)} distinct products).")
    finally:
        connection.close()


if __name__ == "__main__":
    main()
```